

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Implementación de Robótica Inteligente	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez Cesar Torres Huitzil	
Módulo:	Actividad: Mini Reto Semana 7	
Fecha: 25 de mayo del 2026		

Resumen

En este mini reto se implementó una red neuronal, con el objetivo de que el Puzzlebot sea capaz de identificar las 7 diferentes señales en tiempo real, comunicándose a su vez a través de tópicos de ROS para publicar los resultados que observa. La comunicación que se implementa es entre el Puzzlebot y una PC externa, esto por medio de ROS, donde en esta ocasión se publica la imagen capturada por la cámara integrada en el robot.

Primeramente, se utilizó la cámara del Puzzlebot para elaborar el dataset, que consiste de 7 clases diferentes, correspondientes a las señales de tráfico utilizadas. Para cada una de las clases hay alrededor de 500 fotografías en 2 entornos diferentes y así darle un poco más de robustez a la red neuronal. Adicionalmente, se hizo data augmentation y así contar con más fotografías para el entrenamiento. Finalmente, una vez implementado el entrenamiento, se corre en la PC localmente, y los resultados obtenidos fueron bastante satisfactorios, cumpliendo así con los objetivos y permitiendo ver las áreas de oportunidad para el reto final.

Objetivos

El objetivo de este mini reto es desarrollar un sistema de visión artificial integrado con ROS para que el Puzzlebot sea capaz de reconocer e identificar señales de tránsito en tiempo real mediante el uso de una red neuronal procesando las imágenes capturadas por la cámara del robot y comunicando los resultados a través de tópicos de ROS.

Entre los objetivos específicos de este mini reto se encuentra el implementar la comunicación entre el Puzzlebot y una PC externa mediante ROS, utilizando tópicos para publicar imágenes capturadas por la cámara del robot y recibir la información procesada por el sistema de detección. Asimismo el reto busca que se genere y se procese un conjunto de datos de señales de tránsito que permitan entrenar o reentrenar una red neuronal para la identificación de las clases seleccionadas. Consecutivamente, esta actividad busca aplicar las técnicas de data augmentation para aumentar la variabilidad del conjunto de entrenamiento y mejorar la capacidad de generalización del modelo ante cambios de luz, ángulos u orientación.

Finalmente, uno de los retos más importantes de este mini reto es el diseñar, entrenar y evaluar una arquitectura de red neuronal profunda para la clasificación o detección de señales de tránsito y evaluar el desempeño del modelo utilizando métricas como accuracy, precisión, recall, matriz de confusión y otras métricas relevantes que permitan identificar fortalezas y áreas de mejora. Igualmente en este reto se debe comprobar experimentalmente el funcionamiento del sistema mediante una prueba en tiempo real, presentando distintas señales frente a la cámara del robot para verificar que el modelo diseñado sea capaz de reconocerlas correctamente.

Introducción

La visión artificial en los últimos años se ha convertido en una de las herramientas fundamentales dentro de la robótica para interpretar información visual del entorno y tomar decisiones con base a la información visual. En el Puzzlebot la cámara puede utilizarse como sensor principal para identificar elementos relevantes del ambiente, como señales de tránsito, líneas de trayectoria, obstáculos o referencias visuales. Este tipo de sistemas combina percepción, procesamiento de imágenes, aprendizaje automático y comunicación entre nodos, por lo que representa una aplicación práctica de la inteligencia artificial en las áreas robóticas.

Para este reto como se mencionó con anterioridad se busca desarrollar un sistema basado en una red neuronal que sea capaz de detectar señales de tránsito a partir de las imágenes capturadas por la cámara del robot. Es por ello que se puede decir que el reconocimiento de señales de tránsito mediante inteligencia artificial normalmente se apoya en redes neuronales profundas. Hablando específicamente de las redes neuronales, estas son modelos de machine learning que apilan neuronas simples en capas y aprende pesos y sesgos de reconocimiento de patrones de los datos para asignar entradas a salidas [1]. En el caso de las imágenes las primeras capas suelen identificar características simples como bordes,

colores o texturas, mientras que las capas más profundas pueden reconocer formas y patrones más complejos. Este enfoque ha sido especialmente importante en retos de visión por computadora, como clasificación de imágenes, reconocimiento de objetos y detección visual, debido a su capacidad de aprender directamente a partir de datos.

Consecuentemente dentro de las redes neuronales se encuentran las redes neuronales de tipo convolucionales (CNN) las cuales son aquellas que utilizan datos tridimensionales para tareas de clasificación de imágenes y reconocimiento de objetos. A diferencia de otras redes neuronales las redes neuronales convolucionales se distinguen por su mejor desempeño con entradas de señal de imagen, voz o audio. Además estas redes neuronales tienen tres tipos principales de capas que son la capa convolucional, la capa de agrupamiento y la capa totalmente conectada. La capa convolucional es la primera capa de una red convolucional. Con cada capa, la CNN aumenta su complejidad, identificando mayores porciones de la imagen. Las primeras capas se enfocan en características simples, como colores y bordes. A medida que los datos de la imagen avanzan a través de las capas de CNN, se comienzan a reconocer elementos o formas más grandes del objeto, hasta que finalmente se identifica el objeto que se busca reconocer [2].

De igual manera la detección de objetos con IA consiste en identificar qué objeto aparece en la imagen o localizarlo dentro de una bounding box. Uno de los algoritmos de detección más usados es YOLO (You Only Look at Once por sus siglas en inglés), el cual permite la detección como un solo problema de regresión, donde una única red neuronal predice directamente las bounding boxes y las probabilidades de clase a partir de la imagen completa [3]. Por consiguiente uno de los temas más usados en la detección de imágenes y en este reto es el data augmentation y el transfer learning. El data augmentation es una técnica que consiste en generar nuevas versiones de las imágenes originales del dataset mediante transformaciones como rotaciones, desplazamientos, cambios de luz y brillo, entre otras transformaciones. El data augmentation es muy útil puesto que aumenta la diversidad del dataset sin la necesidad de capturar nuevas imágenes. Por otro lado, el transfer learning es una técnica en la que se aprovecha el conocimiento aprendido previamente por una red neuronal entrenada en un conjunto de datos grande y se adapta a una nueva tarea. En lugar de entrenar una red desde cero, se parte de un modelo preentrenado y se ajustan algunas capas para reconocer las nuevas clases de interés. En el caso de este reto, se aplicó transfer learning a partir de un modelo YOLO preentrenado, con el objetivo de adaptar la red neuronal a la detección de las señales de tránsito.

Solución del problema

La solución se diseñó como un pipeline distribuido en cuatro fases secuenciales que parten de la recolección de datos hasta el deployment en el robot:

- **Fase 1 — Dataset:** recolección de imágenes con la cámara CSI del Puzzlebot, etiquetado manual con bounding boxes en Roboflow, preprocesamiento y aumentación.
- **Fase 2 — Entrenamiento:** entrenamiento de YOLO11m sobre el dataset preparado en Google Colab utilizando una GPU NVIDIA T4.
- **Fase 3 — Inferencia:** deployment del modelo (best.pt) en una PC con GPU NVIDIA mediante un nodo de inferencia ROS 2 que procesa los frames recibidos desde el robot.
- **Fase 4 — Integración:** comunicación distribuida vía DDS entre la Jetson del Puzzlebot (publisher de cámara) y la PC (subscriber + procesamiento), compartiendo el mismo ROS_DOMAIN_ID.

La justificación de esta arquitectura distribuida es liberar a la Jetson del cómputo de inferencia para que pueda mantener fluido el control del robot (Pure Pursuit) mientras una máquina con más recursos se encarga del detector pesado.

Recolección y etiquetado

Se recolectaron 3,700 imágenes originales utilizando la cámara CSI del Puzzlebot (sensor IMX219 a resolución nativa de 1640×1232 px). El etiquetado se realizó manualmente en la plataforma Roboflow generando bounding boxes para 7 clases de señales de tráfico relevantes para la navegación autónoma del robot.

Distribución de clases

La distribución de instancias por clase quedó balanceada gracias al control durante la recolección, evitando la necesidad de aplicar pesos por clase (class weights) durante el entrenamiento. Se aplicó Resize: 640×640 como paso de preprocesamiento.

Clase	Número de instancias
ahead_only	536
give_way	524
left_turn	528
right_turn	537

roundabout	536
stop	524
work_ahead	510
Total	3,695

Data augmentation

Se configuró un multiplicador de 3 imágenes generadas por cada imagen original del set de entrenamiento, resultando en aproximadamente 10,000 imágenes de entrenamiento finales. Las transformaciones aplicadas se seleccionaron específicamente para simular las condiciones de operación del Puzzlebot:

Augmentation	Rango	Justificación
Blur (Gaussian)	hasta 1.5 px	Lentes sucios, desenfoque de cámara
Motion Blur	hasta 10 px	Movimiento del robot durante captura
Brightness	$\pm 23\%$	Cambios de iluminación en el entorno
Ruido	Hasta 1.5 px	Lentes sucios, desenfoque de cámara
Rotation	$\pm 10^\circ$	Pequeñas inclinaciones de la cámara

División y exportación

El dataset final se exportó en formato YOLOv11 desde Roboflow con la siguiente división:

- **Train:** 9,444 imágenes (incluye augmentation)
- **Validation:** 556 imágenes (sin augmentation)

Entrenamiento del modelo

Se seleccionó YOLOv11m (medium) como detector base por tres razones:

- Ofrece un balance favorable entre precisión y velocidad de inferencia con sus ~20M de parámetros
- Es la última iteración de la familia YOLO de Ultralytics con mejoras de arquitectura sobre v8 (módulos C3k2 y C2PSA)
- Permite exportación posterior a TensorRT para deployment optimizado en GPU.

Hiperparámetros

El entrenamiento se realizó en Google Colab con GPU NVIDIA T4 (15.6 GB VRAM). Los hiperparámetros se eligieron considerando las limitaciones de hardware del entorno gratuito:

Parámetro	Valor	Justificación
epochs	200	Límite superior; EarlyStopping detiene el entrenamiento al converger
patience	10	Tolera 10 épocas sin mejora antes de detener
imgsz	640	Estándar de YOLO; coincide con el resize hecho en Roboflow
batch	12	Ajustado a los 15.6 GB de VRAM de la T4 (uso ~7 GB)
workers	4	Conservador para Colab gratuito (limitación de CPU/RAM)
optimizer	AdamW	Recomendado por Ultralytics para detección con datasets medianos
lr0 / lrf	0.001 / 0.01	Learning rate inicial y final relativo (cosine decay)
amp	True	Mixed precision; aprovecha los Tensor Cores de la T4
cache	False	Colab free tiene ~13 GB RAM; cache="ram" lo saturaría
save_period	10	Checkpoint cada 10 épocas como respaldo ante cortes de sesión

Resultados del entrenamiento

El modelo convergió rápidamente, alcanzando $mAP_{50} > 0.99$ a partir de la época 12. Los mejores pesos (best.pt) corresponden a la época 37.

Métrica	Valor
mAP@50 (val)	0.994
mAP@50-95 (val)	0.827
Precision (val)	0.987
Recall (val)	0.986
Memoria GPU utilizada	7.07 GB / 14.9 GB

Sistema de inferencia distribuido en ROS 2

La comunicación entre el Puzzlebot y la PC se basa en el middleware DDS de ROS 2. Ambos equipos están conectados a la misma red local y configurados con el mismo ROS_DOMAIN_ID.

Topics y mensajes

Topic	Tipo de mensaje	Publicador
/camera/image_raw/compressed	sensor_msgs/CompressedImage	Jetson (Puzzlebot)
/yolo/detections	vision_msgs/Detection2DArray	PC (yolo_inference_node)
/yolo/image_annotated/compressed	sensor_msgs/CompressedImage	PC (yolo_inference_node)

Se utiliza CompressedImage (JPEG) en lugar de Image cruda para reducir el ancho de banda en la red: un frame de 1640×1232 BGR8 ocupa ~6 MB, contra ~80 KB del mismo frame comprimido como JPEG. Esto permite mantener buenos FPS sin saturar la red WiFi.

Quality of Service (QoS)

Tanto el publicador como el suscriptor utilizan el mismo perfil QoS, requisito indispensable para que DDS permita la comunicación:

```
qos = QoSProfile(  
    reliability=QoSReliabilityPolicy.BEST_EFFORT,  
    history=QoSHistoryPolicy.KEEP_LAST,  
    depth=1,  
)
```

Se eligió BEST_EFFORT (en lugar de RELIABLE) porque en un stream de video es preferible descartar frames perdidos antes que retransmitirlos: un frame retransmitido llegaría desfasado y entorpecería el control en tiempo real. Con depth=1, solo se mantiene el frame más reciente en la cola.

Descripción del código implementado

Nodo de cámara en la Jetson

La Jetson ejecuta un nodo de captura que abre la cámara CSI mediante un pipeline de GStreamer y publica los frames como mensajes CompressedImage. Se aprovecha el encoder JPEG por hardware (NVJPEG) presente en la Jetson para minimizar uso de CPU.

Nodo de inferencia en la PC

Archivo: yolo_inference_node.py. Este es el componente central de la solución. Su diseño desacopla la recepción del stream de cámara de la inferencia mediante dos hilos lógicos: un

callback ligero que solo almacena el último frame recibido, y un timer separado que ejecuta YOLO sobre ese frame a una frecuencia configurable.

Conversión de detecciones a `vision_msgs`

Las predicciones de Ultralytics se traducen al mensaje estándar `Detection2DArray`, lo que permite que otros nodos del ecosistema ROS (por ejemplo, un controlador de comportamiento) consuman las detecciones sin acoplarse a una API específica de YOLO:

```
for (x1, y1, x2, y2), c, k in zip(xyxy_all, conf_all, cls_all):
    det = Detection2D()
    bbox = BoundingBox2D()
    bbox.center.x = float((x1 + x2) / 2.0)
    bbox.center.y = float((y1 + y2) / 2.0)
    bbox.size_x = float(x2 - x1)
    bbox.size_y = float(y2 - y1)
    det.bbox = bbox

    hyp = ObjectHypothesisWithPose()
    hyp.hypothesis.class_id = self.class_names[int(k)]
    hyp.hypothesis.score = float(c)
    det.results.append(hyp)
    det_array.detections.append(det)
```

Optimizaciones de inferencia

- **Half precision (FP16):** se activa con `half=True` en `model.predict()`. Aprovecha los Tensor Cores de la GPU para inferencia $\sim 2\times$ más rápida sin pérdida significativa de mAP.
- **Inferencia desacoplada del callback:** el callback solo guarda el puntero al último frame; la inferencia corre en un timer aparte. Así, si el GPU se ralentiza, no se acumulan frames en cola.
- **Verbose desactivado:** `verbose=False` elimina los prints internos de Ultralytics por cada inferencia, evitando overhead de I/O en consola.
- **conf threshold configurable:** permite ajustar el balance entre falsos positivos y recall sin reentrenar.

Resultados

Los resultados obtenidos fueron los nodos de ROS 2 trabajando en conjunto para realizar la detección de señales de tránsito. Al recibir la imagen de la cámara del puzzlebot, el nodo que procesa el modelo de la red neuronal entrenado muestra la predicción de las clases disponibles cuando aparece una de las señales de tránsito en cámara. Visualmente, el nodo que se ejecuta en una máquina externa se encarga de rodear a la señal de tránsito con una *boundingbox*, marcando la clase predicha y un valor para indicar qué tan seguro está el

modelo que el objeto pertenece a tal clase. Este último valor se encuentra entre 0 y 1, siendo 1 que el modelo está 100% seguro de la predicción.

A continuación, se muestra un video en el que se evidencia el funcionamiento del proyecto: [VIDEO] <https://youtu.be/CzMzVZ37Sho>

Este último consta de 2 secciones. En la primera se muestra cómo se ve visualmente en funcionamiento la detección en un espacio controlado. En este caso, en lugar de poner al robot en la pista junto con las señales, se van mostrando de una en una, o varias señales a la vez para verificar si el modelo es óptimo en esas condiciones. Esa experimentación sirvió como depuración para corregir detalles tanto de los nodos como del entrenamiento mismo. La segunda parte muestra al robot moviéndose a través de distintos puntos de la pista.

Para esta sección se utilizó el paquete de *teleop_twist_keyboard* para controlar el robot desde la PC a través del tópico */cmd_vel*. Al conducir, la cámara va detectando las distintas señales de tránsito exitosamente. El único detalle es que cuando hay una gran distancia entre el robot y la señal de tránsito de “*left_turn*”, confunde esta señal en específico con la clase “*right_turn*”, principalmente por la similitud entre ambas señales. Esto se corregirá posteriormente al añadir más imágenes al dataset para todas las clases y modificando los parámetros del entrenamiento.

Conclusiones

Con base en el desarrollo y los resultados obtenidos en este mini reto, los objetivos planteados fueron cumplidos de manera satisfactoria. Se logró implementar un sistema de visión artificial integrado con ROS 2 capaz de detectar y clasificar señales de tránsito en tiempo real utilizando una red neuronal basada en YOLOv11. Asimismo, se consiguió establecer correctamente la comunicación distribuida entre el Puzzlebot y una PC externa mediante tópicos de ROS, permitiendo la transmisión eficiente de imágenes y la publicación de las detecciones generadas por el modelo. Además, el uso de técnicas de data augmentation y transfer learning permitió entrenar un modelo robusto y con una alta capacidad de generalización, obteniendo métricas de desempeño sobresalientes, como un mAP@50 de 0.994, precisión de 0.987 y recall de 0.986.

De igual manera, se comprobó experimentalmente el funcionamiento del sistema mediante pruebas en tiempo real, donde el robot fue capaz de detectar correctamente distintas señales de tránsito mientras se desplazaba en la pista. Esto demuestra que la arquitectura distribuida

implementada resultó adecuada para liberar la carga computacional de la Jetson y mantener un procesamiento fluido de inferencia en una computadora externa con GPU.

Sin embargo, aunque los objetivos principales se cumplieron, se identificaron algunas limitaciones durante las pruebas. Esto se debe principalmente a la similitud visual entre ambas clases y a la necesidad de contar con una mayor variedad de ejemplos en el dataset bajo diferentes condiciones de distancia, perspectiva e iluminación. Como posible mejora a la metodología implementada, se propone ampliar el conjunto de datos con imágenes en diferentes condiciones de distancia e iluminación. También podrían ajustarse hiperparámetros como el learning rate, el batch size y el número de épocas para mejorar la convergencia del modelo y reducir confusiones entre clases similares. Finalmente, una optimización futura sería implementar inferencia directamente en la Jetson utilizando TensorRT, aumentando así la autonomía y eficiencia del sistema.

Referencias

1. IBM. (s.f.). ¿Qué son las redes neuronales?
<https://www.ibm.com/mx-es/think/topics/neural-networks>
2. IBM. (s.f.). Redes neuronales convolucionales, definidas.
<https://www.ibm.com/mx-es/think/topics/convolutional-neural-networks>
3. Datacamp. (2024). Explicación de la detección de objetos YOLO.
<https://www.datacamp.com/es/blog/yolo-object-detection-explained>